

Solution Requirements

Date	29/09/2026
Team ID	UNM1425EduGenie
Project Name	EduGenie
Maximum Marks	4 Marks

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Authentication	The current EduGenie documentation does not define a user login or authentication feature. Student authentication is listed as a future enhancement .	Low
2	Authorization levels	The current system does not define separate user roles or permission levels. The application is designed primarily for students/learners.	Low
3	External interfaces	EduGenie connects with the Google Gemini API to generate AI-powered educational content. The frontend communicates with the FastAPI backend through HTTP/JSON API requests.	High
4	Transactions processing	The system receives student input, processes the selected learning activity, sends an appropriate prompt to Gemini, receives the generated response, and returns it to the frontend.	High
5	Reporting	The system displays generated educational results such as answers, explanations, quizzes, summaries, and learning recommendations. A student dashboard and progress reports are documented as future enhancements.	High
6	Business rules, etc.	The system must process the selected learning activity and learning level and use the appropriate module for Q&A, explanation, quiz generation, summarization, or learning recommendations.	High
7	Compliance to laws or regulations	The system should protect API credentials and user-provided information. API keys must be stored using environment variables and must not be exposed in frontend code or committed to GitHub.	High
8	Other	The system provides five main learning activities: Question Answering, Topic Explanation, Quiz Generation, Summarization, and Learning Recommendations.	High

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	The application should process student requests efficiently and return generated educational content without unnecessary delay.	Application loads successfully and returns a response within a reasonable time under normal conditions.
2	Scalability	The modular architecture should allow additional users, learning activities, and educational features to be added in the future.	New modules and features can be added without redesigning the complete application.
3	Security & Data Privacy	API keys must be protected using environment variables. Credentials must not be exposed in source code or frontend JavaScript.	GEMINI_API_KEY is stored in .env; .env is excluded from GitHub.
4	Reliability & Availability	The application should handle temporary Gemini API failures and backend errors safely.	Temporary Gemini 503 UNAVAILABLE errors are retried, and appropriate errors are returned when services are unavailable.
5	Usability & Accessibility	The interface should be simple and easy for students to use. Users should be able to select an activity, enter content, generate a response, view the result, and copy it.	Task selection, input, Generate button, result display, and Copy button work correctly.
6	Other – Maintainability	The system should use a modular architecture so that individual learning features can be maintained and extended independently.	Separate modules are maintained for explanation, Q&A, quiz, summary, learning path, and Gemini communication.